

Fast_prime-Claude

cryptografi

Analisis

deskripsi chall :

tidak ada,karena dari ai

source code :

```
#!/usr/bin/env python3
# =====
# MiniFastPrime - CTF Crypto (RSA) - level pemula / SageMath
# =====
# Ide: prima RSA dibuat dengan bentuk khusus
#
#      p = k * M + (e^a mod M)
#      q = l * M + (e^b mod M)
#
# dengan M = perkalian 10 bilangan prima pertama.
# Akibatnya n = p*q == e^(a+b) (mod M) <-- kebocoran!
# =====
import random
from math import prod, gcd
from sympy import primerange, isprime

FLAG = b"CTF{fastprime_mini_sage_ez}"

E = 65537
M = prod(list(primerange(2, 100))[:10]) # 2*3*5*7*11*13*17*19*23*29 =
6469693230

A_MAX = 1000 # 1 <= a, b < 1000
K_MIN, K_MAX = 512, 1024 # 2^9 <= k, 1 < 2^10

def gen_prime():
    """Buat prima M), kembberbentuk k*M + (E^a modalikan p saja."""
    while True:
        a = random.randrange(1, A_MAX)
        k = random.randrange(K_MIN, K_MAX)
        p = k * M + pow(E, a, M)
        if isprime(p) and gcd(E, p - 1) == 1:
            return p
```

```
def chunk(data: bytes, size: int = 8):
    """n kita cuma ~86 bit, jadi flag dipecah per 8 byte."""
    return [int.from_bytes(data[i:i + size], "big") for i in range(0,
len(data), size)]

if __name__ == "__main__":
    p = gen_prime()
    q = gen_prime()
    while q == p:
        q = gen_prime()

    n = p * q
    assert all(m < n for m in chunk(FLAG))

    c = [pow(m, E, n) for m in chunk(FLAG)]

    with open("output.txt", "w") as f:
        f.write(f"n = {n}\n")
        f.write(f"e = {E}\n")
        f.write(f"M = {M}\n")
        f.write(f"c = {c}\n")
    print("output.txt dibuat.")
```

output :

n = 21655349834150020637045959

e = 65537

M = 6469693230

c = [16830406029544597219613071, 10535053366630118895180486,
7366775366649826363523050, 12164704630967835342044421]

di source code yang di berikan oleh claude,kita bisa tau bahwa p dan q di buat dengan fungsi gen_prime.dimana inti dari fungsi ini adalah membuat p dan q dengan mengalikan k dan lalu di tambah dengan hasil E pangkat a yang di moduluskan M

$$p = k \cdot M + (E^a \bmod M)$$

dimana nilai dari a diambil secara acak dari rentang angka 1 sampai 1000 dan k diambil secara acak dari rentang angka 512 sampai 1024.

```
def gen_prime():
    """Buat prima M), kembberbentuk k*M + (E^a modalikan p saja."""
    while True:
        a = random.randrange(1, A_MAX)
        k = random.randrange(K_MIN, K_MAX)
        p = k * M + pow(E, a, M)
        if isprime(p) and gcd(E, p - 1) == 1:
            return p
```

Solusi

jadi karena nilai dari p dan q di dapat dari persamaan $p = k \cdot M + (E^a \bmod M)$ untuk mencari p kita menggunakan discrete logarithm dalam mencari nilai a, kenapa kita bisa mencari a menggunakan discrete logarithm? karena discrete logarithm dibuat untuk mencari nilai x dari $g^x \bmod m = h$ nah nilai x ini adalah eksponen yang kita cari jadi karena kita mencari a (eksponen) jadi kita bisa menggunakan discrete logarithm .untuk mencari nilai a menggunakan discrete logarithm pertama tama kita harus membuat persamaan

$$g^x \bmod m = h$$

jadi untuk membuat persamaan diatas kita bisa menggunakan n.

$$n = p \cdot q$$

$$n = (k_1 M + r_p) \cdot (k_2 M + r_q)$$

$$n = k_1 k_2 \cdot m^2 + k_1 m r_q + r_q k_2 m + r_p r_q$$

$$n \bmod M = (k_1 k_2 \cdot m^2 + k_1 m r_q + r_q k_2 m + r_p r_q) \bmod M$$

$$n \bmod M = 0 + 0 + 0 + r_p r_q \bmod M$$

$$n \bmod M = e^a \cdot e^b \bmod M$$

$$n \bmod M = e^{a+b} \bmod M \quad \text{\#persamaan ini baru bisa kita masukan ke discrete algorithm}$$

jadi dari $n \bmod M = e^{a+b} \bmod M$ kita bisa menggunakannya untuk mencari nilai dari a+b menggunakan discrete algorithm. setelah dapat a+b kita baru mencari nilai a dengan membrute force nya dari rentang 1 sampai nilai dari a+b -1. lalu kita juga harus mencari k dengan membrute forcenya dari 512 sampai $1024 + 1$. setelah p dihitung dan jika $n \bmod p$ hasilnya 0 maka p kita benar.

solver :

```
from sage.all import *
from Crypto.Util.number import long_to_bytes , inverse

n = 21655349834150020637045959
e = 65537
M = 6469693230

ct = [16830406029544597219613071, 10535053366630118895180486,
7366775366649826363523050, 12164704630967835342044421]
```

```

Z_M = Zmod(M)

n_ZM = Z_M(n)
e_ZM = Z_M(e)

s = discrete_log(n_ZM,e_ZM)
if s:
    print(f"nilai a+b = {s}")

t = e_ZM.multiplicative_order()
if s < t:
    print(f"s lu aman bosqu ")
else:
    print("wadoh")

found = False
for a in range(1,s):
    r_p = int(e_ZM**a)
    for k in range(512,1024):
        p_test = k*M+(r_p)

        if p_test > 1 and n%p_test == 0:
            print(f"p = {p_test}")
            p = p_test
            found = True
            break
    if found:
        break

q = n//p
print(f"q = {q}")

phi_n = (p-1)*(q-1)
d = inverse(e,phi_n)

flag = b""
for c in ct:
    m = pow(c,d,n)
    flag += long_to_bytes(m)

print(flag.decode())

```

flag =

```
L$ sage '/mnt/d/py_repin/njajal.py'  
nilai a+b = 1130  
s lu aman bosqu  
p = 5854093791017  
q = 3699180540527  
CTF{fastprime_mini_sage_ez}
```